DP-OPS-0019: Strengthening Doctrine and Establish Context Hazard Guardrails for Library Directories to Prevent Drift

1. Executive Summary

The governance of autonomous and semi-autonomous AI agent systems necessitates a rigorous adherence to "Context Hygiene" to prevent systemic entropy, commonly referred to as "Drift." In the Stela operating system, defined by its constitutional documents (TRUTH.md, AGENTS.md) and operational binaries (ops/), the integrity of the agent's context window—the immediate set of instructions and files defining its reality—is paramount. Recent forensic analysis of the repository state [1] reveals a latent vulnerability: the absence of explicit "Negative Constraints" regarding the inclusion of large library directories (docs/library/agents, docs/library/tasks, and docs/library/skills) within the global context manifest.

"Drift"—the gradual erosion of logic, structure, or intent—often originates not from malicious external actors, but from the internal pollution of the agent's working memory. When an agent is flooded with conflicting persona definitions (e.g., loading the "Security Reviewer" persona while acting as a "Contractor") or excessive procedural data (loading all "Skills" simultaneously), its ability to adhere to the active contract (TASK.md) degrades. This phenomenon is classified as a "Context Hazard."

This report details the research, architectural design, and execution plan for **DP-OPS-0019**, a Dispatch Packet designed to strictly codify "Context Hazards" and establish immutable guardrails around the Library Directories. The proposed intervention transitions the system from a permissive context model to a strict "Just-In-Time" (JIT) retrieval model, enforced by constitutional amendments in TRUTH.md and automated policing via tools/context_lint.sh. By formally designating Library Directories as "Context Hazards," we ensure that agents access tools only when explicitly authorized, preserving the "Context Minimalist" philosophy essential for high-fidelity autonomous operation.

---

2. Theoretical Framework: Context, Memory, and Drift

To understand the necessity of DP-OPS-0019, one must first establish a theoretical framework regarding how Large Language Models (LLMs) interact with file-based context in a repository setting. The Stela system operates on a "Pointer-First" constitution [1], which posits that truth is maintained through unique references rather than duplication. This section explores the mechanics of context, the nature of drift, and the specific dangers of context

pollution.

2.1. The Ontology of Context in Stela

In the Stela architecture, context is defined not as a vague accumulation of history, but as a precise, file-based injection. The document docs/CONTEXT.md [1] explicitly states: "Context is not memory; it is a file. In a stateless system, 'Context' is the precise set of artifacts required to rehydrate an agent... If it's not in the Context, it doesn't exist."

This definition implies a binary reality for the AI agent:

1. **In-Context (The Real):** Files listed in ops/lib/manifests/CONTEXT.md are loaded into the prompt window. They form the "Constitutional Reality" of the session. The agent treats these files as immutable laws and immediate facts.
2. **Out-of-Context (The Void):** Files residing in the repository but not in the manifest (e.g., projects/ or docs/library/ unless referenced) are theoretically accessible but exist outside the immediate cognitive scope of the agent.

The "Context Hazard" emerges when artifacts intended for *reference* (Out-of-Context) are mistakenly migrated to *residency* (In-Context). Industry research on "Context Engineering" supports this distinction, emphasizing that agents perform optimally when they maintain lightweight identifiers and load heavy data only at runtime.[2] The "Context-Aware Monolith" pattern [3] further suggests that while integrated context is beneficial, it must be managed to prevent "over-engineering" or cognitive overload.

2.2. Defining Drift: Semantic vs. Architectural

Drift is a central concern in AI-assisted development. It is defined in Stela's AGENTS.md [1] as "Logic or files not aligned with TRUTH.md coordinates." However, external literature clarifies that drift manifests in two primary forms:

- **Architectural Drift:** Subtle, model-generated design changes that break security invariants or structural rules without violating syntax.[4] For example, an agent might introduce a new directory structure that mimics an existing one but deviates slightly in naming convention, creating a "shadow structure" that bypasses governance tools.
- **Semantic Drift:** A degradation in the agent's understanding of its role or objective.[5] This often occurs when the context window is polluted with conflicting instructions. If an agent reads R-AGENT-06 (Security Reviewer) [1] while tasked with a simple refactor, it may hallucinate security vulnerabilities or adopt a tone of refusal ("I cannot do this") inappropriate for its current "Contractor" role.

2.3. The Context Pollution Vector

Context pollution occurs when irrelevant or conflicting information is forced into the LLM's attention window. In the context of docs/library/, pollution is particularly dangerous because

the artifacts are *authoritative*. They are not random noise; they are "Canon" files (R-AGENT-XX, S-LEARN-XX).

- **Persona Collision:** If docs/library/agents/ is loaded globally, the LLM is simultaneously exposed to the definitions of "Architect," "Integrator," "Cleaner," and "Security Reviewer." The "Open Prompt" [1] assigns the role of "Contractor," but the presence of other high-authority persona definitions in the context can lead to "Role Bleed," where the Contractor begins to mimic the behaviors (or refuse tasks based on the constraints) of other agents.
- **Instruction Dilution:** The skills/ directory contains procedural knowledge (e.g., "How to use the Stela Stack" [1]). If all skills are loaded, the specific procedural instructions in the active TASK.md must compete for attention with the generic instructions in the skills. This dilution increases the probability of the agent ignoring specific "Negative Constraints" (Don'ts) in the DP.

Therefore, the prevention of drift requires the rigorous exclusion of these libraries from the global context manifest. They must be treated as "Hazardous Materials"—useful when needed, but dangerous if left exposed.

---

3. Forensic Audit of the Stela Repository

To design the guardrails for DP-OPS-0019, we must first perform a deep forensic audit of the current repository state, utilizing the provided dump file.[1] This audit identifies the specific surfaces where hardening is required.

3.1. The Constitution (TRUTH.md)

The file TRUTH.md [1] serves as the "System Constitution" and "SSOT" (Single Source of Truth). It currently defines the "4-Line Heuristic" for filing doctrine:

1. Run the system -> ops/
2. Explain the system -> docs/
3. Do the work -> projects/
4. Temporary trash -> storage/

It establishes "Anti-drift invariants," such as "If uncertain: STOP; ask for the missing input." However, a critical gap exists: **There is no explicit prohibition against loading library files into the global context.** The constitution implicitly trusts the Operator to curate the context manifest (ops/lib/manifests/CONTEXT.md) correctly. In a high-churn environment, implicit trust is a failure mode. Without a constitutional amendment defining "Context Hazards," an Operator might accidentally include docs/library/ in a bulk context load, believing it aids the agent, when in fact it precipitates drift.

3.2. The Staffing Protocol (AGENTS.md)

AGENTS.md [1] defines the "Behavioral Logic Standard" for all AI agents. It includes "Operational Directives" like:

- **Anti-Drift Governance:** "Actively monitor for structural anomalies."
- **Reuse-First Discipline:** "Cross-reference all proposals against existing templates."

Crucially, it contains a "Negative Constraint" regarding *creation*: "Do not manually create markdown files in docs/library/skills/." However, it lacks a Negative Constraint regarding *consumption*. There is no directive stating: "Do not load library artifacts unless explicitly requested." This omission means that an Integrator agent, when drafting a new DP, has no rule preventing it from adding docs/library/agents to the requested context list.

3.3. The Library Inventory (docs/library/)

The docs/library/ directory is a repository of high-authority documents. The dump [1] reveals its structure:

Directory	Content Type	Examples	Hazard Potential
docs/library/agents/	Persona Definitions	R-AGENT-01 (Architect), R-AGENT-06 (Security)	**High:** Role confusion. Agent may adopt wrong jurisdiction or authority level.
docs/library/skills/	Procedural Knowledge	S-LEARN-06 (Git Forensics), S-LEARN-05 (Security)	**Medium:** Attention dilution. "Solution" steps in skills may conflict with DP steps.
docs/library/tasks/	Workflow Templates	B-TASK-03 (Integrate), B-TASK-10 (Audit)	**Medium:** Process confusion. Agent may attempt to run a full integration workflow on a minor task.

The file docs/library/INDEX.md [1] acts as the "Authorized Surface Registry." It lists these files as "Protected Canon." This registration is necessary for validity but creates a semantic

ambiguity: if a file is "Canon," should it not be known to the agent? DP-OPS-0019 must clarify that "Registry" does not equal "Residency."

3.4. Existing Tooling Analysis

The repository contains scripts intended to enforce structure, but they require hardening to support the new "Context Hazard" doctrine.

- tools/context_lint.sh [1]:
  - *Current Logic:* Parses ops/lib/manifests/CONTEXT.md and verifies that every listed path exists on disk.
  - *Gap:* It only checks for *presence* (missing files). It does not check for *prohibition* (forbidden files). It acts as an availability monitor, not a compliance guard.
- ops/lib/skill/skill_lib.sh [1]:
  - *Current Logic:* Includes a function context_hazard_check that greps for docs/library/skills/ in the manifest and fails if found.
  - *Opportunity:* This logic is the seed of the solution. However, it is currently buried in a skill-management library. It needs to be elevated to the primary context_lint.sh tool and expanded to cover Agents and Tasks.

3.5. The "Open Prompt" Vulnerability

The ops/bin/open script [1] generates the session prompt. It explicitly lists "CANON POINTERS," including docs/library/MANUAL.md. While MANUAL.md is safe (it's a cheat sheet), the proximity of this pointer to the other library directories increases the risk that an LLM might infer that *all* library directories are valid context. The prompt explicitly says: "Context is not memory; it is a file." This reinforces the need for the file (CONTEXT.md) to be strictly policed.

---

4. Threat Modeling: The "Library Swarm" Attack Vector

To fully justify the rigorous guardrails proposed in DP-OPS-0019, we simulate the failure modes that occur when "Context Hazards" are realized. We term this the "Library Swarm"—a state where the agent is overwhelmed by the collective noise of the library.

4.1. Scenario A: The Security Reviewer Hallucination

- **Trigger:** The Operator adds docs/library/agents/R-AGENT-06.md (Security Reviewer) [1] to ops/lib/manifests/CONTEXT.md.
- **Operation:** The Operator dispatches a simple task: "Refactor the README to fix a typo."
- **The Glitch:** The agent reads AGENTS.md (Role: Contractor) but also reads R-AGENT-06.md (Role: Security Reviewer). The Security Reviewer prompt instructs: "Use PROACTIVELY... Flags secrets, SSRF, injection...".
- **Outcome:** The agent refuses to fix the typo, claiming the README lacks a "Security

Policy" section (hallucinated requirement based on R-AGENT-06 logic). Or, worse, it attempts to "fix" the README by adding irrelevant security disclaimers, constituting **Semantic Drift**.

4.2. Scenario B: The Skill Dilution

- **Trigger:** The Operator adds the entire docs/library/skills/ directory to the context to "make the agent smarter."
- **Operation:** The Operator dispatches a task defined in TASK.md to "Update the CSS for the login button."
- **The Glitch:** The context now contains S-LEARN-02 (Stela Stack), S-LEARN-05 (Security Protocols), S-LEARN-06 (Git Forensics).
- **Outcome:** The agent's attention span is exhausted by processing the "Procedures" and "Drift Preventers" of seven different skills. It misses the specific instruction in TASK.md to "Use the primary color variable." Instead, it applies a generic CSS fix or, triggered by S-LEARN-05, starts auditing the login form for Zod validation [1], scope-creeping the task into a backend refactor.

4.3. Scenario C: The Task Recursion

- **Trigger:** The Operator adds docs/library/tasks/B-TASK-03.md (Integrate Command) [1] to the context.
- **Operation:** The Operator requests a "Plan" using B-TASK-04.
- **The Glitch:** The agent sees B-TASK-03's definition of "Integration" (integrator -> code-reviewer -> security-reviewer).
- **Outcome:** Instead of outputting a simple plan, the agent hallucinates a full "Integration Report" structure, filling it with fictitious "Handoff Artifacts" because B-TASK-03 requires them. The agent prioritizes the *library definition* of a task over the *immediate request* of the Operator.

These scenarios demonstrate that context pollution is not merely inefficient; it is actively destructive to the governance model. It replaces the "Active Contract" (TASK.md) with a "Confused Parliament" of conflicting library definitions.

---

5. Proposed Doctrine: The "Just-In-Time" (JIT) Context Model

To mitigate these threats, DP-OPS-0019 introduces a shift in doctrine: the **"Just-In-Time" (JIT) Context Model**.

5.1. Philosophy: Kernel vs. Tools

The system context will be conceptually divided into two tiers:

1. **The Kernel (Global Context):** These files *must* be present in ops/lib/manifests/CONTEXT.md. They define the "Operating System" of the agent.
    - TRUTH.md (Constitution)
    - AGENTS.md (Identity/Logic)
    - SoP.md (Time/State)
    - TASK.md (Mission)
2. **The Tools (JIT Context):** These files *must not* be present in the global manifest. They reside in docs/library/. They are loaded *only* when a Dispatch Packet explicitly lists them in the "Required Context Load" section.
    - docs/library/agents/*
    - docs/library/skills/*
    - docs/library/tasks/*

5.2. The "Context Hazard" Designation

We formally designate the Library Directories as "Context Hazards." This term is chosen for its semiotic weight. A "Hazard" is not just "unnecessary"; it is "dangerous." This terminology is aligned with the "Context Hazard" warning already present in SKILL.md.[1] DP-OPS-0019 elevates this from a warning in a markdown file to a rule in the Constitution (TRUTH.md).

5.3. Negative Constraints

The security architecture of Stela relies heavily on "Negative Constraints" (what the agent *must not* do). We will implement a new Negative Constraint in docs/CONTEXT.md and AGENTS.md: **"Do not include Library Directories in the Global Context."** This constraint will be enforced technically by the linter, preventing the Operator (or an AI drafting a PR) from committing a polluted manifest.

---

6. Implementation Specification (The "Hardening")

This section details the specific changes required to the repository files to implement the JIT Context Model.

6.1. Constitutional Amendment (TRUTH.md)

We will amend TRUTH.md [1] to explicitly define the Context Hazard doctrine. The current "Structure" section lists "Docs library (curated)" but does not restrict its usage.

- **Action:** Add a "Context Hazard" subsection under "2. CONSTITUTION (INVARIANTS + ANTI-DRIFT)."
- **Text:** "The Library (docs/library/{agents,tasks,skills}) is for *reference*, not *residency*. Never add library directories to the global context manifest. They are invoked JIT

(Just-In-Time) by the DP."

6.2. Behavioral Logic Update (AGENTS.md)

We will update AGENTS.md [1] to bind the AI agents to this new doctrine.

- **Action:** Add "Context Hygiene" to the "Operational Directives" in Section 2.
- **Text:** "Context Hygiene: Verify ops/lib/manifests/CONTEXT.md is free of library artifacts (Agents, Tasks, Skills). Treat their inclusion as a critical system failure (Context Hazard)."

6.3. Manifest Defense (docs/CONTEXT.md)

We will update the explanatory document docs/CONTEXT.md [1] to align with the new doctrine.

- **Action:** Add a "Negative Constraints" section.
- **Text:** "Context Hazards: The following paths MUST NOT be included in the Global Context: docs/library/agents/, docs/library/tasks/, docs/library/skills/. These are loaded only by specific Dispatch Packets."

6.4. Automated Policing (tools/context_lint.sh)

The shell script tools/context_lint.sh [1] is the enforcement arm. We will modify it to grep for the forbidden paths.

Current Logic (Simplified):

Bash

```
# Checks if files exist
for path in "${required_artifacts[@]}"; do
  if [[! -e "${target}" ]]; then fail...; fi
done
```

New Logic (Hardening):

Bash

```
# Checks for Hazards
forbidden_paths=("docs/library/agents" "docs/library/tasks" "docs/library/skills")
```

```
for hazard in "${forbidden_paths[@]}"; do
  if grep -q "$hazard" "$MANIFEST_PATH"; then
    fail "CONTEXT HAZARD DETECTED: '$hazard' must not be in the global context."
  fi
done
```

This change converts the linter from a passive check into an active firewall.

---

7. Draft Dispatch Packet: DP-OPS-0019

The following is the complete, strictly formatted Dispatch Packet to be placed in TASK.md, utilizing the templates and context provided in the dump.[1]

DP-OPS-0019: Context Hazard Guardrails

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

- **Base Branch:** work/proposal-protocol-2026-02-03
- **Required Work Branch:** work/context-hazard-2026-02-03
- **Base HEAD:** 64cfae6ed (or Current)

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING

ANYTHING)

Worker must confirm loaded before acting:

- TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - tools/context_lint.sh
  - docs/CONTEXT.md

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

II) SCOPE & SAFETY

- **Objective:** Strengthen doctrine and establish context hazard guardrails for library directories (agents, tasks, skills) to prevent drift.
- **Non-Goals (optional, drift-killer):**
  - No changes to projects/.
  - No changes to individual Skill or Agent definitions (files inside the directories).

Target Files allowlist (hard gate)

- TRUTH.md
- AGENTS.md
- docs/CONTEXT.md
- tools/context_lint.sh
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.

- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The Library directories (docs/library/agents, tasks, skills) define specialized capabilities.
- **Drift:** Currently, there is no explicit barrier preventing these directories from being added to the global context manifest (ops/lib/manifests/CONTEXT.md).
- **Desired State:** "Context Hazards" are formally defined in Canon and technically enforced by the context linter.

B) REQUEST

Numbered tasks (concrete, no vibes):

1. **TRUTH.md:** Add "Context Hazard" doctrine to Section 2 (Constitution). Define Library directories as JIT-only resources that must not reside in the global context.
2. **AGENTS.md:** Update Section 2 (Behavioral Logic) to include a "Context Hygiene" directive enforcing the absence of hazards in the manifest.
3. **docs/CONTEXT.md:** Update Section 1 (The Manifest) to explicitly list "Negative Constraints" (what must NOT be in the manifest).
4. **tools/context_lint.sh:** Implement the "Context Hazard Guard." Add logic to grep the manifest for docs/library/agents, docs/library/tasks, and docs/library/skills. Fail if found.
5. **SoP.md:** Log the governance update.

Required Content:

- In TRUTH.md: Use the term "Context Hazard" explicitly.
- In tools/context_lint.sh: The failure message must be "FAIL: CONTEXT HAZARD: must not be in the global context."

C) CHANGELOG

- **Log (human-readable, 1–6 bullets):**
  - Defined "Context Hazard" in TRUTH.md (Library directories are JIT-only).
  - Updated AGENTS.md with Context Hygiene directive.
  - Hardened tools/context_lint.sh to fail if agents, tasks, or skills are found in the global manifest.

    - Updated docs/CONTEXT.md with negative constraints.

D) PATCH / DIFF

- **Format:** Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
    - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
    - Prefer: bash tools/context_lint.sh
    - **Logic Test:** Temporarily add a hazard (e.g., docs/library/skills/S-LEARN-01.md) to ops/lib/manifests/CONTEXT.md and verify the linter FAILS. Remove it and verify it PASSES. Record this toggle in RESULTS.
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
    - Prefer: bash tools/lint_truth.sh

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/skill/skill_lib.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution. Do not edit the Provenance block.
- Promote: run ops/lib/skill/skill_lib.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include SKILL.md.
- Proof required in RESULTS:
    - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" SKILL.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0019-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)

- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0019-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied (above). (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording; copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

---

8. Deep Dive: Implementation Logic and Rationale

The following analysis details the rationale behind the specific technical decisions made in DP-OPS-0019.

8.1. Logic for tools/context_lint.sh Modification

The modification of tools/context_lint.sh is the "Hard Gate" of this proposal. Currently, the script operates in a "Positive Verification" mode—it checks if required files exist.[1] We are adding a "Negative Verification" mode.

The proposed logic introduces a forbidden_paths array. By iterating through this array and grep-ing the manifest, we create a deterministic failure state. This is superior to complex parsing or regex matching because the CONTEXT.md format is simple (backticked paths). A simple substring match is robust, fast, and easy to audit. This aligns with the "Stela Standard"

of "Boring is good" found in CONTRIBUTING.md.[1]

8.2. Semantic Consistency in AGENTS.md

The update to AGENTS.md [1] leverages the "Behavioral Logic Standard." By defining "Context Hygiene" as a behavioral standard, we enlist the AI's own logic processing to polic itself. When an AI (acting as Integrator) reads AGENTS.md, it internalizes the rule "Do not load library artifacts." This creates a "Self-Correction" layer that operates *before* the code is even written or the linter is run. It moves the governance "left" in the development lifecycle.

8.3. Registry (INDEX.md) vs. Residence (CONTEXT.md)

A key insight from the forensic audit is the confusion between the "Registry" and "Residence." docs/library/INDEX.md [1] lists all library files as "Authorized Surface Registry." This gives them legitimacy. However, legitimacy does not imply ubiquity. The DP clarifies that INDEX.md is the "Phone Book" (who exists), while CONTEXT.md is the "Meeting Room" (who is present). DP-OPS-0019 ensures that only the essential staff (TRUTH, AGENTS, TASK) are permanently in the meeting room, while specialists (Security Reviewer, Git Forensics Skill) are called in only when needed.

8.4. Impact on Future Scalability

This JIT model is critical for scalability. As the library grows to include hundreds of skills (S-LEARN-100) or dozens of agents, a "Load Everything" approach would exceed even the largest context windows or, more likely, degrade the reasoning quality of the model due to the "Lost in the Middle" phenomenon. By enforcing JIT loading, we ensure that the Stela system can scale indefinitely in terms of *capability* (Library size) without degrading in terms of *performance* (Context size).

---

9. Verification and Rollback Strategy

The DP mandates a rigorous verification process, utilizing the "Red Team" methodology described in the receipt section.

9.1. The "Logic Test" (Red Teaming)

The verification step requires the worker to *deliberately break* the system to prove the guardrail works.

1. **Pollute:** The worker manually adds docs/library/skills/S-LEARN-01.md to ops/lib/manifests/CONTEXT.md.
2. **Verify Failure:** The worker runs tools/context_lint.sh. The expected result is FAIL: CONTEXT HAZARD....
3. **Clean:** The worker reverts the change to CONTEXT.md.

4. **Verify Success:** The worker runs tools/context_lint.sh. The expected result is OK.

This proof is essential. Without it, we have only "Paper Governance"—rules that are written but potentially functionally inert. This active testing ensures "Technical Governance."

9.2. Rollback Plan

If the DP causes unforeseen friction (e.g., blocking valid workflows that rely on implicit context):

1. **Revert:** git revert <merge_commit>.
2. **Restore:** Use ops/bin/dump to restore the previous state of tools/context_lint.sh.
3. **Log:** Update SoP.md with the rollback event and reason.

However, given that this DP imposes a *restriction* rather than a *capability*, the risk is primarily one of "developer annoyance" (having to manually add skills to DPs) rather than "system breakage." This friction is intentional and desired—it forces the Operator to be explicit about context requirements.

---

10. Strategic Implications

DP-OPS-0019 is a pivotal maturation point for the Stela repository. It transitions the system from "Early Stage" (where flexibility and ease of access are prioritized) to "Growth Stage" (where structure, governance, and hygiene are prioritized). By codifying "Context Hazards," we align Stela with high-assurance software principles found in safety-critical systems [6], where the boundaries of the system must be rigorously defined and enforced.

This doctrine prepares Stela for "Agentic AI" workflows [7], where multiple autonomous agents might collaborate. In a multi-agent system, context pollution is fatal—it causes agents to hallucinate each other's roles. By enforcing strict context boundaries, DP-OPS-0019 lays the foundation for a robust, multi-agent future where each agent operates with a clean, task-specific view of reality.

---

11. Conclusion

The "Context Hazard" represents a significant, if invisible, threat to the integrity of the Stela system. By allowing the "Library" to bleed into the "Constitution," we risk Semantic Drift and Role Confusion. DP-OPS-0019 provides a comprehensive solution:

1. **Legal:** Defines the hazard in TRUTH.md.
2. **Social:** Enforces hygiene in AGENTS.md.

3. **Technical:** Blocks pollution in tools/context_lint.sh.

This triad of controls ensures that Stela remains a high-fidelity environment for autonomous software development. The Dispatch Packet drafted in Section 7 is ready for immediate execution, subject to Operator approval and the rigorous verification steps outlined therein.

Status: Ready for Review

Risk: Low (Governance Hardening)

Impact: High (Drift Prevention)

End of Report

Works cited

1. dump-full-work-proposal-protocol-2026-02-03-64cfae6ed.txt
2. Effective context engineering for AI agents - Anthropic, accessed February 3, 2026, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
3. Stop guessing: 5 AI architecture patterns that actually prevent project failure - PAELLADOC, accessed February 3, 2026, https://paelladoc.com/blog/stop-guessing-5-ai-architecture-patterns/
4. I Built a 2300-File Codebase with AI. Here's the Jig I Built to Prevent Architectural Drift. | by Stefan van Egmond | Jan, 2026 | Medium, accessed February 3, 2026, https://medium.com/@stefanvanegmond/i-built-a-2300-file-codebase-with-ai-heres-the-jig-i-built-to-prevent-architectural-drift-56453fe2d5b4
5. Avoid AI code drift with small, well-defined steps - Henrik Jernevad, accessed February 3, 2026, https://henko.net/blog/avoid-ai-code-drift-with-small-well-defined-steps/
6. Where AI Assurance Might Go Wrong Initial lessons from engineering of critical systems, accessed February 3, 2026, https://arxiv.org/html/2502.03467
7. Seizing the agentic AI advantage - McKinsey, accessed February 3, 2026, https://www.mckinsey.com/capabilities/quantumblack/our-insights/seizing-the-agentic-ai-advantage